Final Engineering Report: Deterministic Market-Data Warehouse

Project Overview

This report details the architectural and semantic engineering of the deterministic market-data warehouse. The system is designed to ingest, validate, and serve historical options and equities data. The primary objective is to construct an operationally realistic repository that prioritizes data lineage, reproducibility, and the deterministic interpretation of vendor feeds over naive sanitization.

Architecture Overview

The warehouse relies on deterministic local analytics workflows utilizing DuckDB-backed parquet access. Raw data is treated as an immutable raw zone. Transformed and validated layers are deterministically generated from this log to ensure stable query ordering. Compute is embedded to provide inspectable local execution and reproducible query behavior, reflecting an operationally cautious design.

Profiling & Data Characterization

System engineering prioritized rigorous empirical profiling before validation-policy design. We operated on the principle that the validation logic must map to the empirical reality of the vendor feed, rather than idealized market mechanics. Aggressive validation initially quarantined a large percentage of option-chain rows, revealing that assumed OHLC invariants were incompatible with observed vendor-feed semantics.

Profiling revealed significant vendor-feed ambiguity. Many irregularities initially classified as syntactic errors were, upon deeper characterization, semantic anomalies reflecting upstream market state representations. The data exhibited behaviors indicative of vendor snapshot semantics rather than strict trade-bar semantics, demanding an operationally cautious interpretation of market-feed behavior.

Validation Philosophy

The validation architecture implements deterministic curation semantics.

Our initial assumption was that OHLC violations (e.g., Low > High, Close outside High/Low range) strictly implied data corruption. However, observed reality invalidated this premise:

- The system encountered widespread systematic violations of OHLC invariants.
- Strict enforcement of these invariants resulted in unrealistic quarantine rates.
- Profiling identified pre-open indicative vendor behavior and snapshotting artifacts that naturally produce these conditions.

Final decision: Preserve uncertain rows with annotations. Instead of destructive filtering, the system utilizes trust annotations. We apply a selective quarantine strategy, isolating only low-ambiguity structural anomalies. Ultimately, only 6 rows were quarantined, while 24k+ rows were preserved with annotations. This approach preserves lineage visibility and defers the decision of filtering to the query layer, empowering downstream consumers to specify their own tolerance for semantic ambiguity.

Systematic annotations include:

- `PREOPEN_INDICATIVE`
- `OHLC_ANOMALY`
- `DUPLICATE_OPTIONS_SNAPSHOT`
- `REORDERED_TIMESTAMPS`

Warehouse & Access Design

The warehouse exposes data through curated parquet representations that explicitly include validation annotations. Curated outputs preserve deterministic ordering semantics to ensure stable downstream retrieval behavior across repeated pipeline executions. Consumers access the data via embedded DuckDB queries, filtering on annotation columns (e.g., `WHERE NOT OHLC_ANOMALY`) as their operational constraints dictate. This decouples the validation pipeline from access-pattern enforcement, maintaining deterministic outputs and reproducible query behavior.

Access Contract Semantics

This section explicitly describes the deterministic retrieval behavior and edge-case handling semantics governing the access layer. These wrappers provide deterministic analytical access semantics only and do NOT implement predictive trading models.

`get_features()`

The `get_features()` function enforces deterministic retrieval semantics for continuous spot and futures datasets. For early-session timestamps where trailing data is insufficient to compute robust windows, it strictly surfaces what is empirically available. It guarantees no fabrication of unavailable rolling-window statistics. The contract dictates that only derivable fields are returned, ensuring downstream consumers operate on causally sound boundaries without synthetic artifacts polluting the analytical state.

`get_features_batch()`

Designed to support vectorized consumer inputs, `get_features_batch()` guarantees the preservation of request ordering. The function implements explicit surfacing of unavailable timestamps (returning nulls or equivalent gap indicators) to maintain the integrity of the output shape. This provides deterministic downstream alignment and ensures the strict avoidance of silent row dropping. This semantic transparency prevents unintended state misalignment in consumer pipelines.

`get_signals()`

The `get_signals()` function exposes point-in-time state for options chains. It employs nearest deterministic snapshot resolution, tying analytical queries to the most recent causally valid vendor state. The implementation enforces a strict "no interpolation" policy between options snapshots. By refusing to guess intermediary pricing states, it ensures the avoidance of synthetic market-state generation. The consumer receives the deterministic observation rather than a continuous mathematical approximation.

Benchmarking

Benchmarking focused on query latency against the local Parquet storage layer.

Benchmark	Iterations	Avg (ms)	P50 (ms)	P95 (ms)	P99 (ms)	Min (ms)	Max (ms)
spot_point_lookup	50	14.310	14.006	17.014	18.698	12.293	19.519
options_snapshot_lookup	50	16.550	16.413	18.798	19.865	14.070	20.793
futures_point_lookup	50	14.990	14.551	17.035	18.017	12.959	18.077
vix_range_query	50	14.919	14.674	16.661	20.032	12.741	22.250

Observed latency distributions demonstrate narrow latency clustering across query classes. This suggests that fixed DuckDB/parquet overhead dominated query cost more than dataset selectivity at the provided assignment scale. The test parameters emphasize warm-process local execution, where filesystem cache effects substantially smooth out read penalties. Consequently, the small dataset behavior highlights framework initialization latency rather than structural retrieval constraints.

These measurements characterize deterministic local analytics behavior only and do NOT imply distributed scalability claims. Scaling strategies are explicitly out of scope for the current design iteration, maintaining our focus on deterministic local execution.

Correctness Boundaries

This section explicitly defines what the validation layer guarantees, what remains outside the current scope, and what assumptions remain weak. These represent deliberate scope boundaries, not accidental oversights, maintaining an operationally cautious approach to data characterization.

Timestamp Assumptions

The system treats vendor timestamps as the ground truth for chronological state transitions, under the assumption that vendor timestamps are causally meaningful. However, this assumption remains structurally weak against upstream clock drift. Temporally shifted but structurally valid rows may evade detection entirely. The pipeline validates the provided physical ordering but cannot dynamically audit the true physical reality of the recorded timestamp.

Structural vs. Financial Correctness

The validation layer prioritizes structural coherence over economic realism. It ensures the mathematical bounds of the data frame (e.g., OHLC integrity, strictly positive bid/ask structures) are explicitly annotated. However, implied-volatility realism is not validated. Deeply out-of-the-money options pricing anomalies or irrational volatility skew structures will pass inspection as long as they meet raw structural constraints.

Cross-Asset Consistency

There is currently no economic consistency validation between futures, spot, and options pricing layers. The pipeline evaluates each stream in isolation. Consequently, an options derivative pricing tick that implies an overt arbitrage violation against the underlying spot asset will not trigger a validation anomaly.

Futures Expiry Transitions

Contract lifecycle behavior is inferred entirely from vendor structure. Expiry-roll semantics remain operationally sensitive and are not dynamically mapped against external exchange calendars. The system deterministically processes the provided instrument symbols but delegates the economic interpretation of physical expiry transitions to the consumer layer.

MLflow & Reproducibility

Reproducibility-first engineering is enforced via MLflow. MLflow is utilized to ensure deterministic run tracking, validation artifact logging, and reproducible warehouse metadata. This benchmark traceability guarantees that any analytical output can be reproducibly reconstructed from the immutable raw zone given the historical pipeline parameters.

Limitations

The system's boundaries reflect deliberate assignment-scale tradeoffs prioritizing inspectability and deterministic reproducibility over raw throughput.

Scale Bottlenecks

The current storage architecture relies heavily on coarse parquet scanning. The lack of partition-aware pruning dictates that queries scan broader dataset swaths than strictly necessary. Furthermore, options-chain cardinality explosion presents a significant scaling risk; the combinatorial expansion of strikes and expiries will aggressively degrade query performance without more granular partitioning strategies.

Memory Limitations

The ingestion and sorting mechanisms incorporate monolithic sequential sorting. This design provides deterministic output but introduces strict in-memory execution boundaries. As the historical data window expands, attempting to process the entire dataset in a single local dataframe will inevitably trigger out-of-memory failures.

Benchmark Limitations

The current latency characterization suffers from a strong warm-cache bias, reflecting repeated access patterns rather than realistic cold-start penalties. Furthermore, the test suite incorporates no concurrency testing and offers no distributed execution characterization. The system is profiled purely as a single-tenant local engine.

Validation Limitations

While effective for pipeline integrity, the annotation taxonomy is incomplete. The system executes no advanced derivative-pricing validation capable of flagging subtle market-microstructure violations. This lacks institution-scale semantic coverage, deferring more complex cross-asset arbitrage checks to future iterations.

Conclusion

The engineering of this warehouse demonstrates that operationally realistic market-data systems must prioritize observation over assumption. By shifting from destructive filtering to trust annotations, the architecture accommodates vendor-feed ambiguity and widespread systematic violations without sacrificing

deterministic outputs or lineage preservation. The resulting system provides a verifiable, reproducibility-first foundation for deterministic local analytics.